DAT158 ML Assignment 2 - Steam Game Price Predictor

Gruppe 7

Jørgen Oltedal

Johannes Tollessen Rosberg
Filip Olav Ulland
Daniella Stensland

30. Oktober 2026

Introduction and problem definition..... 3
Data collection and exploration.....3
Dataset and initial plan.....3
Exploration and cleaning.....3
Data preparation and first models.....5
Train, validation, and test data..... 5
Features and encoding.....5
Baseline and model comparison..... 6
Final model and evaluation..... 7
Website.....8
Limitations and possible improvements..... 8
Conclusion.....9
Use of AI.....9

Introduction and problem definition

Our goal is to build a machine learning model that estimates the price of Steam games, from information about the game listing. In this supervised regression problem, the input is a set of game attributes, and the target is the game listing's price in USD.

We used mean absolute error (MAE) to score our models, as it gives an easy to understand average prediction error in dollars. We also measured MAE separately for different price ranges. Early models performed well overall, but consistently underestimated games priced above $20. Improving these high price predictions became the main priority, even if overall MAE increased slightly.

We originally envisioned this as a fun tool to check how overpriced a game is, by comparing the estimated price with the game's listed price, rather than determining the objective value of a game.

Data collection and exploration

Dataset and initial plan

We used "Steam games complete dataset" from kaggle. This dataset contains game prices, alongside related information such as genres, user tags, reviews, achievements, release dates, publishers, and developers. Originally, we planned to include review statistics, game details, achievements, genres, tags, and release dates. This set of features changed during development.

The source CSV contained over 40,800 records. After cleaning, the dataset used for the final model contained 10,959 games. Free games and games priced less than $0.9 were excluded, as the goal is predicting the prices of paid games.

Exploration and cleaning

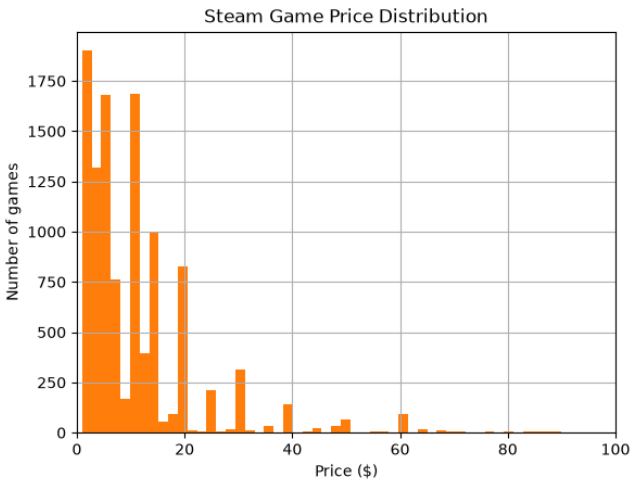
We started by examining the raw dataset to decide which features to keep, and how to process the raw information into usable values. We then created a cleaning script that generated a clean dataset. The cleaning script excluded all "bundle" entries, leaving only entries classified as Steam apps. For price, we used the original game price when available, otherwise falling back to the discounted price, or removing the entry if no price is listed. Price and review statistics were converted to numerical values, and release dates to release years. We also removed duplicate Steam app IDs and entries with unusable values.

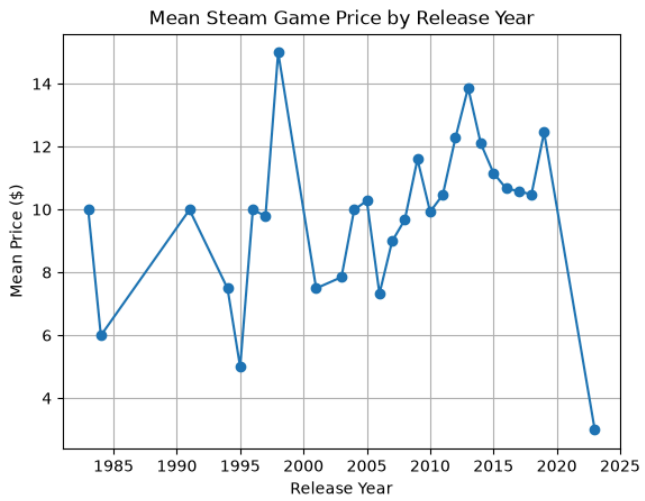
We excluded free to play games, as they often rely on alternate monetization methods, making them less comparable to games with an up-front price. We also excluded games priced at $0.9 and below, as the large number of very cheap games could introduce a bias toward lower price predictions, affecting the model's ability to estimate higher priced

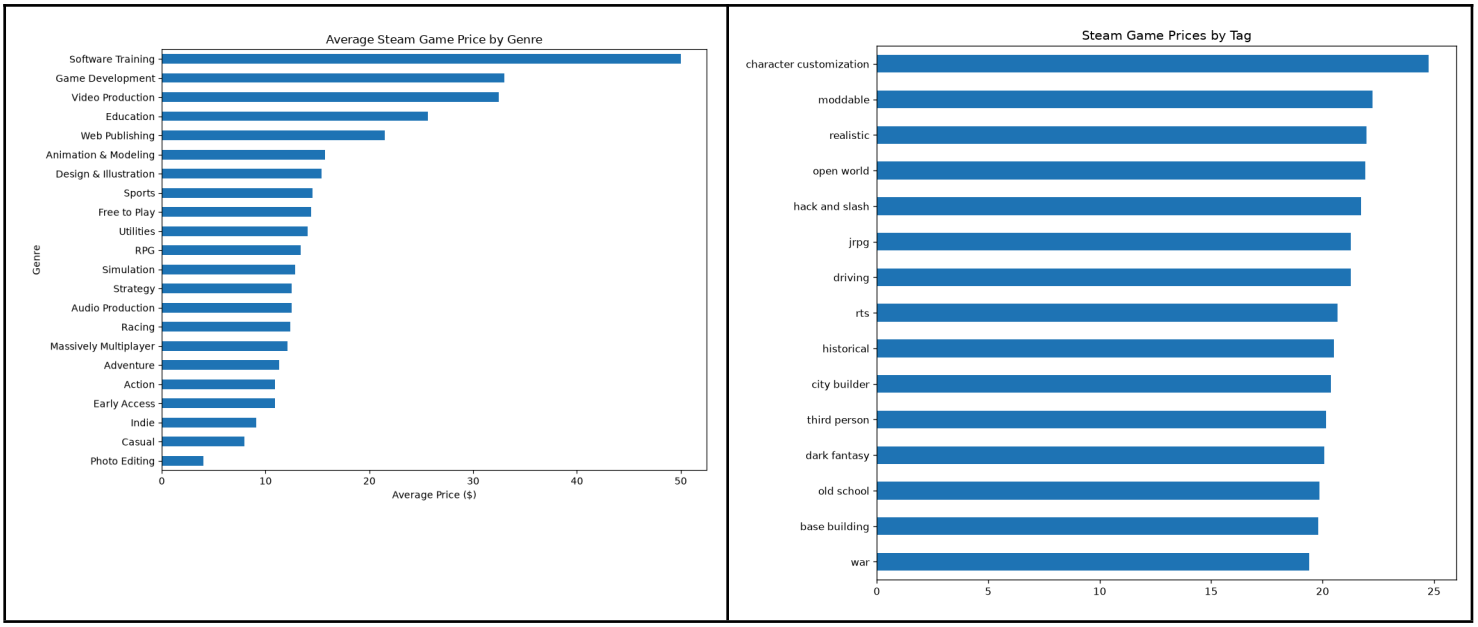
games. Finally, we excluded games priced above $90 after finding multiple suspiciously high prices, as well as odd repeated prices.

After cleaning the dataset, we explored the price distribution and compared prices across release years, genres, and tags. We found that the majority of games were cheap, while expensive games were much less common. This imbalance became relevant when evaluating the models. We also saw a clear relationship between genres and tags, and price.

To better understand the dataset, we created several graphs showing how game prices were distributed, and how they varied across different features. The figures below show the overall price distribution, as well as price by genre, tag, and release year. Since the dataset contained too many tags to display clearly, we limited the tag graph to the 15 highest priced tags. The remaining tags followed the same gradual decrease in average price, reaching just above $5 at the lowest. While most games are relatively cheap, prices vary considerably between genres and tags. This suggested these features could be useful for predicting prices.







Data preparation and first models

Train, validation, and test data

We split the 10,959 games into 70% training (7,671), 15% validation (1,644), and 15% testing (1,644), using random_State=42. The training data was used to prepare model features, i.e. encoding categorical features and scaling numerical features. Validation data was used to measure and compare the models. Test data was kept untouched until we chose the final model, letting us evaluate it based on data that had not been seen during development.

Features and encoding

The final model used popular tags, genre, game details, publisher, developer, languages, achievements, and release year. Features with comma separated categorical values contained in strings were converted to lists and encoded using MultiLabelBinarizer, which represents the features as an array of binary inputs for all the possible entries. The two numerical features were scaled using StandardScaler. We combined the encoded categories and numerical features into a single set of features. Most games only have a small number of the possible categorical features, so we used a format that did not store all the zero values. Encoders and scalers were fitted on training data, and reused for validation and testing.

Review counts and positive-review percentages are present in the cleaned dataset, but not in the final set of features. This was an oversight in feature selection.

Baseline and model comparison

We started with a DummyRegressor that always predicted the median training price. This gave us a baseline to compare our models against.

The first model we chose was Ridge Regression, as it works well with numerical and encoded features. We then trained a Random Forest Regressor, to check whether a nonlinear model could capture more complex relationships between features and prices. Both models outperformed the baseline, with Random Forest achieving the lowest validation MAE. This suggested the selected features were useful for predicting prices.

At this point, we also started measuring MAE by price range to see where the model made its errors. This revealed that the model was good at estimating lower price games, but made large errors with more expensive titles, with an MAE of $50.08 for games priced at $40 and above. We suspected that the uneven price distribution and suspicious/unreliable high prices in the dataset contributed to this problem. Refining the dataset reduced overall MAE from $7.01 to $5.59, while the $40+ MAE decreased to $34.89. However, this was partly because the new dataset contained fewer extreme prices to pull the MAE up, so the MAEs cannot be directly compared.

Random Forest performed much better on training data than validation data. In an early experiment, it achieved a training MAE of $2.82 and a validation MAE of $5.69, suggesting the model was overfitting the training data. We experimented with hyperparameter tuning, but this only improved expensive game estimates slightly. This combined with the expensive game errors led us to explore different models.

During these experiments, we also expanded our feature selection to include developer, publisher, and supported languages. Our assumption was that developer and publisher information could help the model distinguish between larger commercial games and smaller indie games. The supported languages could also give an indication of a game's production scale, as larger companies should have more resources to translate their games. We updated the cleaning script and encoded these new features in the same way as genres and tags.

We then tested several other models, including Extra Trees, Gradient Boosting, Support Vector Regression, LightGBM, CatBoost, and XGBoost. While most of them performed well overall, most still struggled with expensive games. Therefore, we experimented with higher weights for expensive games, to make their errors more important to the model. Weighted XGBoost gave the most promising results, reducing the $40+ validation MAE to $22.33, compared to Random Forest's $31.11. This came at the cost of slightly worse overall MAE.

We decided to continue using weighted XGBoost, and used hyperparameter tuning to improve it further. We tested 36 different configurations. For each configuration, we divided the training data into three parts, trained the models on two parts, and evaluated it on the remaining part. We repeated this three times, using a different part for evaluation each

time, and compared the average results. We ranked the configurations primarily by their MAE on games priced at $20 and above, while also considering overall MAE. The best configuration used 700 trees, a maximum depth of 5, a learning rate of 0.04, and training weights of 3 for games priced $20–40 and 6 for games priced $40 and above. This reduced the $40+ validation MAE to $20.40, with an overall MAE of $5.69.

We did further experiments attempting to improve the model. We tried combining predictions from XGBoost and Random Forest, hoping this would improve overall accuracy without losing too much accuracy on expensive games. However, XGBoost performed better on its own based on our priorities. We also tried removing remaining suspicious entries, 117 in total, but found that this made expensive game predictions worse, and decided to keep the original dataset.

Validation results on the final dataset are shown below. The models are compared using the same split. Lower MAE is better.

Model	**Overall MAE**	**$40+ MAE**
Median baseline	$7.58	—
Ridge Regression	$5.47	—
Random Forest	$5.35	$31.11
Weighted Random Forest	$5.54	$24.07
Unweighted XGBoost	$5.34	$26.76
Tuned weighted XGBoost	$5.69	$20.40

Final model and evaluation

We chose the tuned weighted XGBoost as our final model and named it Steamduck20, after its approximately $20 MAE for games priced $40 and above. Once we finished comparing and tuning the models, we evaluated Steamduck20 on the test data that had been kept separate throughout development.

Metric	**Validation**	**Final Test**
Overall MAE	$5.69	$5.53
$20+ MAE	$13.62	$12.38
$20-40 MAE	$10.38	$9.78
$40+ MAE	$20.40	$19.64

The test results were slightly better than the validation results. Final overall MAE was $5.53 across 1,644 test games. This suggests that the model is consistent on unseen data.

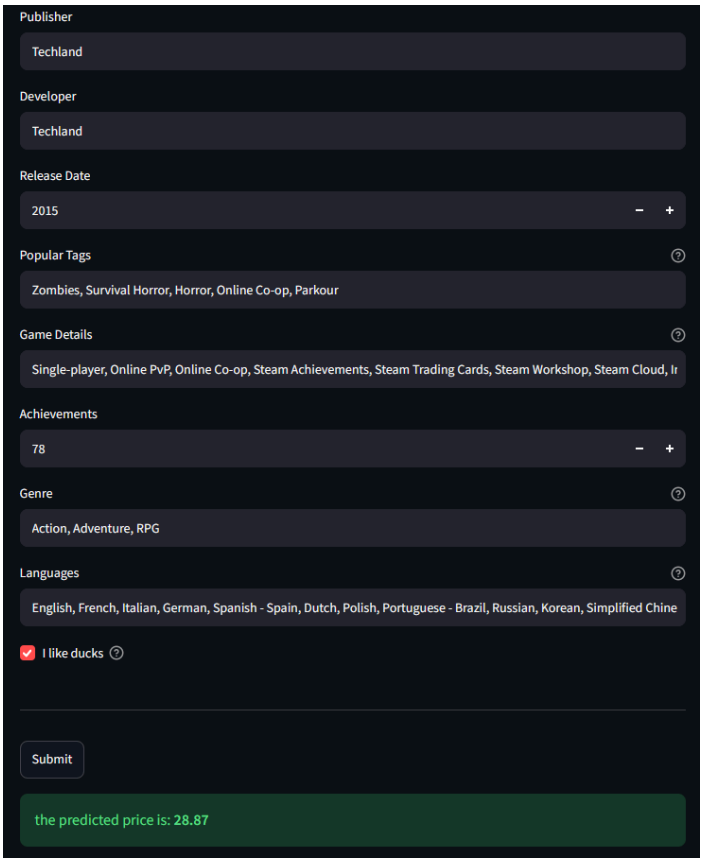
The largest problem was still expensive games. For the 43 test games priced at $40 or above, the actual average was $58.35, while the model's predictions averaged $38.99. The model still underestimates the price of expensive games, but performs considerably better than our early Random Forest models.

Website

We chose to develop the website using Python and host it on Streamlit at https://bonusducks.streamlit.app. The inputs were initially based on the cleaned dataset and changed throughout development to fit the model inputs.

The website allows users to input information about a Steam game. When the form is submitted, the website uses the trained Steamduck20 model to estimate the game's price in USD, and displays the result.

The image shows the website, with details for Dying Light entered. The predicted price is $28.87, while the actual price is $24.99. Techland clearly isn't charging enough for this gem.



Limitations and possible improvements

We originally planned to use review statistics and included them in the cleaned dataset, but accidentally left them out when preparing the features for the models. We discovered this after evaluating Steamduck20. A later experiment suggested that review statistics could help, but we kept the selected model unchanged.

The model was also affected by the uneven price distribution, with few expensive games available for training and evaluation. Some prices in the dataset appeared unreliable, such as game bundles seemingly being listed as games, and listed prices have undoubtedly changed in the 7 years since the dataset was scraped from Steam. These factors limit how accurately it is possible to estimate the price of an individual game.

We included supported languages as separate categorical features, similar to genres and tags, hoping this would help distinguish larger productions from smaller indie games. However, this made the website less practical to use, as users would need to enter every supported language for a game. In hindsight, the number of supported languages might

have provided much of the same information, while requiring only a single numerical input. We did not test this alternative. Manually inputting genres, tags, and game details is also a big ask. However, these features are the core basis for price prediction, and we would lose too much information by simplifying their inputs.

Many entries in the dataset only had a discounted price listed. We used this price with the assumption that the discounted price might simply represent the game's regular price, since none is listed. However, we did not verify this assumption, and we could have introduced unnecessary inconsistencies in our training data.

Conclusion

Throughout the project, we followed the main steps of the ML project lifecycle: understanding the problem, examining and cleaning the data, preparing features, comparing models, tuning the most promising model, and finally evaluating it on test data. We found that overall MAE did not describe overall performance well, so we also compared errors across price ranges. Steamduck20 achieved a final overall MAE of $5.53 and a $40+ MAE of $19.64. Although it still underestimated expensive games, it performed much better than our earlier models for our goal.

Our original idea was to make a fun tool for checking whether a game might be overpriced by comparing its listed price to the model's estimate. Since Steamduck20 tends to underestimate expensive games, it ends up suggesting that many of them are overpriced. While this is a limitation of the model rather than proof that the games are actually overpriced, it does align with our subjective opinion that many games are too expensive. In that sense, our original idea was a success, although not quite in the way we intended.

Use of AI

We used ChatGPT as an assistant throughout the project. It helped us identify potential inconsistencies and suspicious values in the dataset that we had not noticed ourselves. We also used it to explore alternate machine learning models, generate repetitive code, and locate and fix bugs.

AI was used to help structure the report, improve the wording, and make technical explanations clearer. Any generated content was reviewed and adjusted to reflect the work we actually performed. The decisions about which experiments to conduct, which results to prioritize, and which model to use were made by the group.

Sources

Antonov, A. (n.d.). Steam games complete dataset [Dataset]. Kaggle. https://www.kaggle.com/datasets/trolukovich/steam-games-complete-dataset